

Debugging

Learning Objectives

- Identify common types of programming errors: syntax, runtime, and logic errors
- Use systematic debugging techniques to locate and fix bugs
- Apply defensive programming practices to prevent errors proactively

Engage (5-7 minutes)

Even experienced programmers spend up to 50% of their time debugging. A bug is like a typo in a recipe—sometimes obvious, sometimes hidden deep in the logic. Finding it requires patience, systematic elimination, and the right tools. Today we learn professional debugging techniques that will make you a more effective programmer.

Explore (10-12 minutes)

Examine this buggy code: a function that should return the average of a list but returns incorrect values. Read it line by line with a partner and list every issue you spot—off-by-one errors, incorrect operators, missing edge case handling. Then fix the bugs and test with different inputs, including an empty list. How did you approach finding each bug?

Explain (10-12 minutes)

Programming errors fall into three categories. Syntax errors violate language rules and prevent compilation—like missing parentheses or incorrect indentation. Runtime errors occur during execution, such as dividing by zero or accessing an index out of range. Logic errors produce incorrect results without crashing—the hardest to find. Debugging techniques include print debugging (adding output statements), using a debugger to step through code, rubber duck debugging (explaining the problem out loud), and binary search debugging (checking the middle of your code to narrow down the issue).

Elaborate (8-10 minutes)

Take a pre-written program with three intentional bugs—one syntax, one runtime, and one logic error. Use print statements to track variable values, add breakpoints in a debugger to step through execution, and write test cases to isolate the logic error. Then practice defensive programming by adding input validation, try-except blocks, and assertions to prevent errors before they occur. Write a function that handles all edge cases gracefully.

Evaluate (5-8 minutes)

Given this code with a logic error: `def find_max(lst): max_val = 0; for x in lst: if x > max_val: max_val = x; return max_val`, identify the bug, explain why it fails for negative numbers, and write the corrected version. Describe your debugging process step by step.